

Servicios externos y seguridad

El sistema deja de estar solo: le pide datos a otro programa por internet, se identifica para hacerlo y desconfía de todo lo que recibe.

Rubén Schnettler • Docente

INACAP Valparaíso • Analista Programador

BLOQUE 5

RECUPERACIÓN

Dónde quedamos



Lo que ya funciona

- Clases que representan el negocio
- Una base de datos con sus tablas
- El CRUD completo, con el SQL separado
- Errores atrapados y entradas validadas



LO QUE FALTA

Todo el dato que hay en el sistema lo escribió una persona. Ningún sistema real trabaja así: **pide datos a otros servicios** —el valor de la UF, un correo, un mapa, un pago— y para eso hay que salir a internet.

Salir a internet trae dos problemas nuevos: el servicio puede fallar, y hay que identificarse para usarlo.

BLOQUE

01

Un programa que le pregunta a otro

Qué es una API, y las cuatro ideas de HTTP que hacen falta

CONCEPTO

API: la puerta por la que un sistema atiende a otro

API • INTERFAZ DE PROGRAMACIÓN DE APLICACIONES

Conjunto de direcciones que un sistema publica para que **otros programas** le pidan datos o le encarguen operaciones. No es para personas: no devuelve una página, devuelve datos.

SERVICIO	QUÉ SE LE PIDE	PARA QUÉ SIRVE EN UN SISTEMA DE GESTIÓN
mindicador.cl	El valor del dólar, la UF o la UTM de hoy	Convertir montos sin que nadie los escriba a mano
Servicio de correo	Enviar un mensaje	Avisar que se aprobó una solicitud
Pasarela de pago	Cobrar y confirmar	Registrar el pago sin manipular la tarjeta
Servicio de mapas	La dirección de un punto	Validar una dirección de despacho

La regla que ordena todo el bloque: **lo que otro ya resuelve bien, no se programa de nuevo**. Se consume.

LO MÍNIMO DE HTTP

Cuatro conceptos que ordenan el resto

La dirección La **URL**: qué servicio y qué recurso.
`https://mindicador.cl/api/dolar`

El método GET pide, POST envía, PUT modifica, DELETE borra

El código Un número que dice **cómo salió**: 200, 404, 500

El cuerpo Los datos que vuelven, casi siempre en **JSON**



ES EL MISMO CRUD, POR LA RED

GET es el *read*, POST el *create*, PUT el *update* y DELETE el *delete*. Lo que se aprendió sobre la base de datos vuelve a aparecer, ahora entre dos programas distintos.

LOS CÓDIGOS DE ESTADO

Qué significa el número que devuelve el servicio

CÓDIGO	SIGNIFICA	DE QUIÉN ES LA CULPA	QUÉ HACE EL PROGRAMA
200	Salió bien	—	Sigue adelante con los datos
400	La petición está mal armada	Del programa que pide	Revisar los parámetros
401 • 403	No autenticado • sin permiso	Del programa que pide	Revisar la clave o el token
404	Ese recurso no existe	Del programa que pide	Avisar que no se encontró
500 • 503	El servicio falló o no está	Del servicio	Reintentar más tarde, o seguir sin ese dato

La familia dice casi todo: **2xx** salió bien • **4xx** es culpa de quien pide • **5xx** es culpa del servicio. Distinguirlos define qué hacer.

EL FORMATO DE LA RESPUESTA

JSON: la misma estructura que un diccionario

Lo que devuelve el servicio

```
{
  "version": "1.7.0",
  "autor": "mindicador.cl",
  "codigo": "dolar",
  "nombre": "Dólar observado",
  "unidad_medida": "Pesos",
  "serie": [
    { "fecha": "2026-08-19T04:00:00.000Z",
      "valor": 952.43 }
  ]
}
```

{ } Un objeto. En Python llega como **diccionario**

[] Una lista. En Python llega como **lista**

"clave" Siempre entre comillas dobles

null Ausencia de valor. En Python es None

Por eso se lee igual que cualquier estructura de Python:

```
datos["serie"][0]["valor"]
```

EJERCICIO 1 • EN VOZ ALTA 6 minutos

Leer la respuesta

```
{  
  "codigo": "uf",  
  "nombre": "Unidad de fomento",  
  "serie": [  
    { "fecha": "2026-08-19", "valor": 39142.51 },  
    { "fecha": "2026-08-18", "valor": 39138.02 }  
  ]  
}
```

Escriba la expresión de Python que obtiene cada cosa, suponiendo que la respuesta está en la variable **datos**.

- 1 El nombre del indicador
- 2 El valor de hoy
- 3 La fecha del segundo registro
- 4 Cuántos registros trae la serie

RESPUESTA • EJERCICIO 1 Revisión en plenario

Las cuatro expresiones

```
1 datos["nombre"]  
  # "Unidad de fomento"  
  
2 datos["serie"][0]["valor"]  
  # 39142.51  
  
3 datos["serie"][1]["fecha"]  
  # "2026-08-18"  
  
4 len(datos["serie"])  
  # 2
```

- 1 El primer nivel es un diccionario: se entra por la clave.
- 2 **serie** es una **lista**: se entra por posición, y el más reciente va primero.
- 3 Cada elemento de la lista es otro diccionario. Se encadenan los accesos.
- 4 **len()** funciona igual que con cualquier lista de Python.

BLOQUE

02

La librería requests

Pedirle datos a un servicio, en cuatro líneas

INSTALACIÓN

requests no viene con Python

Terminal, con el entorno activado

```
(.venv) $ pip install requests  
Successfully installed requests-2.32.3  
(.venv) $ pip freeze > requirements.txt
```



POR QUÉ EL ARCHIVO REQUIREMENTS.TXT

Deja escrito **qué hace falta instalar** para que el proyecto funcione. Sin él, quien clone el repositorio no sabe qué le falta.

pip El instalador de paquetes de Python

requests La librería estándar de hecho para hablar HTTP

pip freeze Lista lo instalado, con su versión exacta



CON EL ENTORNO ACTIVADO

Si no aparece `(.venv)` al principio de la línea, la librería se instala en el Python del sistema y el proyecto no la encuentra.

SE ESCRIBE EN PANTALLA

El primer llamado, línea por línea

src/indicadores.py

```
A import requests
B URL = "https://mindicador.cl/api/dolar"
C respuesta = requests.get(URL, timeout=5)
D print(respuesta.status_code)
E datos = respuesta.json()
F print(datos["serie"][0]["valor"])
```

- A Se importa la librería instalada.
- B La dirección, en una constante. No se repite por todo el código.
- C `get()` hace la petición y espera. `timeout=5` corta a los 5 segundos.
- D El código de estado. Si no es 200, lo que sigue no sirve.
- E `.json()` convierte el texto de la respuesta en diccionarios y listas de Python.
- F Se entra a la estructura como en el ejercicio anterior.

SE EJECUTA EN PANTALLA

Qué devuelve

Terminal

```
$ python src/indicadores.py  
200  
952.43
```

Dos líneas: el código de estado y el valor del día. El dato no lo escribió nadie: **llegó de afuera**.

.status_code El número: 200, 404, 500...

.json() La respuesta convertida a estructuras de Python

.text La respuesta en crudo, como texto

.ok True si el código es menor que 400

DETALLE TÉCNICO

Parámetros: no se pegan a mano en la dirección

Frágil

```
url = ("https://api.ejemplo.cl/buscar?" +  
      "nombre=" + nombre +  
      "&area=" + area)  
r = requests.get(url)
```

Si el nombre trae un espacio o una tilde, la dirección queda mal formada.

Correcto

```
r = requests.get(  
    "https://api.ejemplo.cl/buscar",  
    params={"nombre": nombre, "area": area},  
    timeout=5  
)
```



LO MISMO QUE EN SQL

params es al HTTP lo que el signo de interrogación es al SQL: **el dato viaja como dato**, y la librería lo codifica.

EJERCICIO 2 • INDIVIDUAL 10 minutos

Su primer llamado

ENUNCIADO

Escriba un archivo que consulte

`https://mindicador.cl/api/uf`, imprima el **nombre** del indicador, el **valor de hoy** y la **fecha**, y que además muestre un mensaje distinto si el código de estado no es 200.

Con `timeout`. Todavía sin `try`: eso viene en el bloque siguiente.

**Qué entregar**

- El archivo que ejecuta
- La salida en la terminal
- Qué pasa si escribe mal la URL

RESPUESTA • EJERCICIO 2

Revisión en plenario

Una solución posible

```
import requests

URL = "https://mindicador.cl/api/uf"

respuesta = requests.get(URL, timeout=5)

if respuesta.status_code != 200:
    print("El servicio no respondió correctamente:",
          respuesta.status_code)
else:
    datos = respuesta.json()
    ultimo = datos["serie"][0]
    print(datos["nombre"])
    print(ultimo["fecha"][:10], ".", ultimo["valor"])
```

- 1 Se comprueba el código **antes** de leer los datos: si el servicio falló, `.json()` también fallaría.
- 2 **ultimo** guarda el primer elemento para no repetir la cadena de accesos tres veces.
- 3 `[:10]` recorta la fecha a **2026-08-19** y deja fuera la hora.

BLOQUE

03

Cuando el servicio no responde

La red falla, y un sistema serio lo tiene previsto

EL PUNTO DE PARTIDA

Las cinco formas de que esto falle

QUÉ PASA	QUÉ LEVANTA REQUESTS	QUÉ DEBERÍA HACER EL SISTEMA
No hay conexión a internet	ConnectionError	Avisar y seguir con lo que tenga guardado
El servicio demora demasiado	Timeout	Cortar y usar el último valor conocido
La dirección está mal escrita	MissingSchema	Es un error de programación: corregirlo
El servicio devuelve 500	nada , hay que revisarlo	Reintentar más tarde
Devuelve 200 pero sin el campo esperado	KeyError	Validar antes de usar el dato



LAS DOS ÚLTIMAS SON LAS PELIGROSAS

No levantan excepción por sí solas. Un 500 con cuerpo vacío y una respuesta a la que le falta un campo **pasan como si todo estuviera bien** hasta que revientan más adelante.

SE ESCRIBE EN PANTALLA

El llamado con sus defensas

```
import requests
def obtener_dolar():
    A    try:
    B        r = requests.get("https://mindicador.cl/api/dolar",
                             timeout=5)
    C        r.raise_for_status()
    D        serie = r.json().get("serie", [])
    E        if not serie:
            return None
        return serie[0]["valor"]
    F    except requests.Timeout:
        print("El servicio demoró demasiado")
    except requests.ConnectionError:
        print("Sin conexión a internet")
    except requests.RequestException as e:
        print("Error al consultar el servicio:", e)
    return None
```

- A Todo el llamado va dentro del `try`.
- B El `timeout` es obligatorio: sin él, el programa puede quedarse esperando para siempre.
- C `raise_for_status()` convierte un 404 o un 500 en excepción.
- D `.get("serie", [])` devuelve lista vacía si la clave no viene, en vez de reventar.
- E Serie vacía devuelve `None`: el que llama decide.
- F Del error más concreto al más general, igual que con la base de datos.

CRITERIO DE DISEÑO

Degradar antes que interrumpir

Se interrumpe

```
valor = obtener_dolar()
total = monto / valor # None → error
```



EL SISTEMA COMPLETO SE DETIENE

Porque un servicio de terceros no respondió. El usuario no puede hacer *nada*, ni siquiera lo que no dependía de ese dato.

Degrada

```
valor = obtener_dolar()
if valor is None:
    valor = repo.ultimo_valor_conocido()
    print("Valor referencial del último día")
total = monto / valor
```



EL SISTEMA SIGUE, AVISANDO

Usa el último valor guardado y **le dice al usuario que es referencial**. Lo que no se puede hacer es usarlo en silencio.

EJERCICIO 3 • EN PAREJA 12 minutos

Ponerle defensas a este llamado

```
def buscar_comuna(codigo):  
    r = requests.get("https://api.ejemplo.cl/comunas/" + codi  
    return r.json()["nombre"]
```

 Corrija cuatro cosas

- Falta el `timeout`
- El código se pega a la dirección
- No se revisa el estado de la respuesta
- Se accede a una clave que puede no venir

**Y UNA QUINTA**

Decida **qué devuelve** la función cuando algo falla, y déjelo escrito.

RESPUESTA • EJERCICIO 3

Revisión en plenario

La función con sus defensas

```
def buscar_comuna(codigo):  
    try:  
        r = requests.get("https://api.ejemplo.cl/comunas",  
                          params={"codigo": codigo},  
                          timeout=5)  
        r.raise_for_status()  
        return r.json().get("nombre")  
    except requests.RequestException as e:  
        print("No se pudo consultar la comuna:", e)  
        return None
```

- 1 El código viaja en `params`: la librería lo codifica y no rompe la dirección.
- 2 `raise_for_status()` hace que un 404 entre al `except`.
- 3 `.get("nombre")` devuelve `None` si la clave no viene, en vez de `KeyError`.
- 4 La función **siempre devuelve algo**: el nombre o `None`. Quien llama sabe qué esperar.

BLOQUE

04

Identificarse ante el servicio

Claves, tokens y el error que termina en la portada del diario

CONCEPTO

Por qué un servicio pide identificarse

MECANISMO	CÓMO VIAJA	DÓNDE SE USA
Sin autenticación	Nada. La dirección es pública	Datos abiertos: indicadores, feriados, clima
Clave de API	Una cabecera o un parámetro con una clave fija	Servicios gratuitos con límite de uso
Token Bearer	La cabecera Authorization: Bearer ...	Lo más común hoy. El token caduca
Usuario y contraseña	Se intercambian una vez por un token	El inicio de sesión del propio sistema



QUÉ HACE LA CLAVE

Le dice al servicio **quién está pidiendo**. Con eso controla el límite de uso, cobra si corresponde y puede cortar el acceso. Por lo mismo, una clave filtrada es **una cuenta ajena en manos de otro**.

EL ERROR MÁS COSTOSO

Dónde NO va una clave

Nunca

```
API_KEY = "sk_live_7f3a9c1e42b8"

r = requests.get(URL, headers={
    "Authorization": f"Bearer {{API_KEY}}"
})
```



QUEDA EN EL HISTORIAL PARA SIEMPRE

Aunque después se borre la línea, **el commit anterior la conserva**. Hay robots que recorren GitHub buscando exactamente ese patrón.

Así sí

```
import os

API_KEY = os.environ.get("API_KEY")
if not API_KEY:
    raise RuntimeError("Falta la variable API_KEY")
```



LA CLAVE VIVE FUERA DEL CÓDIGO

En una **variable de entorno**. El código dice cómo se llama, nunca cuánto vale.

HÁGALO AHORA

El archivo .env y el .gitignore

```
# .env – NO se sube al repositorio
API_KEY=sk_live_7f3a9c1e42b8
BASE_URL=https://api.ejemplo.cl
```

```
# .gitignore
.env
*.db
.venv/
__pycache__/
```

```
# .env.example – este SÍ se sube
API_KEY=
BASE_URL=https://api.ejemplo.cl
```



POR QUÉ EL EJEMPLO

Quien clone el repositorio necesita saber **qué variables hacen falta**, sin conocer sus valores. El archivo de ejemplo trae los nombres y deja los valores en blanco.

SE ESCRIBE EN PANTALLA

El cliente autenticado, completo

```
import os, requests
class ClienteAPI:
    A def __init__(self):
        self.__clave = os.environ.get("API_KEY")
        self.__base = os.environ.get("BASE_URL")
    B def __cabeceras(self):
        return {"Authorization": f"Bearer {{self.__clave}}"}
    C def obtener(self, recurso, **parametros):
        try:
            r = requests.get(f"{{self.__base}}/{{recurso}}",
                            headers=self.__cabeceras(),
                            params=parametros, timeout=5)
            r.raise_for_status()
            return r.json()
        except requests.RequestException as e:
            print("Error consultando", recurso, e)
            return None
```

- A La clave se lee **una vez** y queda privada. Conviene además abortar aquí mismo si falta.
- B Privado: arma la cabecera. Nadie fuera toca la clave.
- C Un método sirve para cualquier recurso: **todo el HTTP vive aquí adentro**.

EJERCICIO 4 • EN PAREJA 8 minutos

Encuentre los tres riesgos

```
import requests

TOKEN = "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9"

def traer_empleados():
    r = requests.get("http://api.ecotech.cl/empleados",
                    headers={"Authorization": TOKEN})
    print("Respuesta completa:", r.text)
    return r.json()
```

Tres problemas de seguridad

- Uno está en la línea 3
- Uno está en la dirección
- Uno está en lo que se imprime

RESPUESTA • EJERCICIO 4

Revisión en plenario

Los tres riesgos

N.º	RIESGO	POR QUÉ IMPORTA	CORRECCIÓN
1	Token escrito en el código	Queda en el repositorio y en todo el historial de commits	Variable de entorno, leída con <code>os.environ</code>
2	La dirección es <code>http</code>	El token viaja sin cifrar: cualquiera en la misma red lo lee	Exigir <code>https</code> siempre
3	Se imprime la respuesta completa	Los datos personales terminan en la consola y en los registros	Imprimir solo lo necesario, nunca el cuerpo entero

Falta además el `timeout` y el manejo de errores, que ya se vieron. Los tres riesgos aparecen juntos en casi todo ejemplo copiado sin revisar.

BLOQUE

05

Desconfiar de todo lo que entra

Del usuario y también del servicio: nada llega
validado

EL CRITERIO

Dos fuentes, el mismo tratamiento



Lo que escribe el usuario

- ¿Viene? • obligatorio
- ¿Tiene la forma correcta? • formato
- ¿Está dentro de lo posible? • rango
- ¿Es del tipo esperado? • conversión

Ya se validó así antes de escribir en la base de datos.



Lo que devuelve el servicio

- ¿El código fue 200?
- ¿Vienen las claves que se esperaban?
- ¿El valor es del tipo esperado?
- ¿Está dentro de un rango razonable?



EL QUE SE OMITE CON FRECUENCIA

Se asume que el servicio siempre responde bien. Un servicio puede cambiar su formato sin avisar, o devolver null un martes cualquiera.

SE ESCRIBE EN PANTALLA

Validar la respuesta antes de guardarla

```
def extraer_valor(datos):  
    A if not isinstance(datos, dict):  
        return None  
    B serie = datos.get("serie")  
    if not serie or not isinstance(serie, list):  
        return None  
    C valor = serie[0].get("valor")  
    if valor is None:  
        return None  
    D try:  
        valor = float(valor)  
    except (TypeError, ValueError):  
        return None  
    E if valor ≤ 0 or valor > 1_000_000:  
        return None  
    return valor
```

- A Primero, que sea lo que se espera. Si el servicio devolvió una lista o un texto, no se sigue.
- B Presencia y tipo de la clave, en una sola comprobación.
- C Que el valor exista. `.get()` no revienta si falta.
- D Conversión defensiva: un número puede llegar como texto.
- E **Rango razonable.** Un dólar a cero o a un millón es un dato roto, aunque sea un número válido.

EJERCICIO 5 • INDIVIDUAL

10 minutos

Validar antes de guardar

ENUNCIADO

Un servicio devuelve el registro de un empleado: `{"rut": ..., "nombre": ..., "horas_mes": ...}`. Escriba la función que lo **valida** y devuelve el objeto **Empleado**, o **None** si algo no cuadra.

**Qué revisar**

- Que vengan las tres claves
- Que el nombre no esté vacío
- Que las horas sean número
- Que estén entre 0 y 240

RESPUESTA · EJERCICIO 5

Revisión en plenario

Una solución posible

```
def a_empleado(registro):  
    if not isinstance(registro, dict):  
        return None  
    for clave in ("rut", "nombre", "horas_mes"):  
        if clave not in registro:  
            print("Falta el campo", clave)  
            return None  
    nombre = str(registro["nombre"]).strip()  
    if not nombre:  
        return None  
    try:  
        horas = float(registro["horas_mes"])  
    except (TypeError, ValueError):  
        return None  
    if not 0 ≤ horas ≤ 240:  
        return None  
    return Empleado(registro["rut"], nombre, horas)
```

- 1 El **for** sobre las claves evita repetir tres veces la misma comprobación.
- 2 **.strip()** descarta un nombre de puros espacios.
- 3 La conversión va en **try**: el servicio puede mandar el número como texto, o como **null**.
- 4 La comparación $0 \leq \text{horas} \leq 240$ se lee como en matemáticas.

BLOQUE

06

Todo junto: **servicio y base**

El dato llega de afuera, se valida y se guarda

EL FLUJO COMPLETO

Cuatro pasos, cada uno con su responsable

1 Pedir · ClienteAPI
Hace la petición con su clave, su timeout y sus excepciones atrapadas.

2 Validar · una función propia
Comprueba presencia, tipo y rango, y devuelve un objeto o None.

3 Guardar · Repositorio
Escribe en la base con consulta parametrizada.

4 Informar · el programa principal
Le dice al usuario qué se trajo, qué se guardó y qué se descartó.

Ninguna de las cuatro clases sabe lo que hacen las otras por dentro. Eso es **bajo acoplamiento**, y es lo que permite cambiar el servicio sin tocar la base.

SE ESCRIBE EN PANTALLA

El programa principal

```
from cliente_api import ClienteAPI
from empleado_repositorio import EmpleadoRepositorio
from validacion import a_empleado
api, repo = ClienteAPI(), EmpleadoRepositorio()
datos = api.obtener("empleados", area="Desarrollo")
if datos is None:
    print("No se pudo sincronizar. Se trabaja con lo guardado.")
else:
    guardados, descartados = 0, 0
    for registro in datos:
        empleado = a_empleado(registro)
        if empleado is None:
            descartados += 1
        else:
            repo.guardar(empleado)
            guardados += 1
    print(f"Guardados: {{guardados}} · descartados: {{descartados}}")
```

Terminal

```
$ python main.py
Guardados: 18 · descartados: 2
```



LOS DOS DESCARTADOS TAMBIÉN SON INFORMACIÓN

Un sistema que dice cuántos registros descartó permite **ir a buscar por qué**. Uno que los descarta en silencio esconde el problema.

BLOQUE

07

La IA y la seguridad del código

Qué revisar cuando el código que consume servicios no lo escribió usted

EL CRITERIO DE LA ASIGNATURA

Lo que pide el criterio 3.1.4

CRITERIO 3.1.4

«Ajusta la seguridad del código desarrollado con apoyo de herramientas de IA, **documentando y justificando las decisiones técnicas adoptadas.**»

QUÉ REVISAR	LA MARCA DEL CÓDIGO SIN REVISAR
Credenciales	La clave escrita dentro del archivo, o en un comentario
Protocolo	http: // en lugar de https: //
Timeout	Ausente: el llamado puede quedarse esperando para siempre
Manejo de errores	Sin try, o con except: pass
Uso de la respuesta	Se entra a las claves sin comprobar que existan

Son cinco, se revisan en menos de un minuto, y están casi siempre.

EJERCICIO 6 • EN PAREJA 12 minutos

Revisar y documentar

```
# Generado por una herramienta de IA
import requests

def enviar_reporte(datos):
    key = "api_5f4dcc3b5aa765d61d8327deb882cf99"
    r = requests.post(
        "http://reportes.ejemplo.cl/api/v1/enviar",
        json=datos,
        headers={"X-API-Key": key}
    )
    return r.json()["id"]
```

**Complete la tabla del informe**

- Aspecto revisado
- Problema detectado
- Decisión: adoptar, modificar o descartar
- Fundamento técnico

RESPUESTA • EJERCICIO 6

Revisión en plenario

La tabla que va al informe

ASPECTO	PROBLEMA	DECISIÓN	FUNDAMENTO
Credenciales	La clave está escrita en el código	Modificar	Queda en el historial del repositorio
Protocolo	Usa http	Modificar	La clave y los datos viajan sin cifrar
Timeout	No hay	Modificar	El programa puede quedar bloqueado
Manejo de errores	Sin try	Modificar	Cualquier fallo de red detiene el sistema
Uso de la respuesta	Entra a ["id"] sin comprobar	Modificar	Si el servicio no lo devuelve, levanta KeyError

La estructura del llamado sirve y **se conserva**. Lo que se reescribe es todo lo que tiene que ver con seguridad y con fallos, que es lo que la herramienta no consideró.

BLOQUE

08

La evaluación de esta parte

Lo que se suma al programa que ya tiene base de datos

EL ENCARGO

Lo que hay que agregar al proyecto

#	CRITERIO	DÓNDE SE DEMUESTRA
3.1.1	Programa scripts que consumen servicios externos con librerías oficiales	La clase ClienteAPI
3.1.2	Usa mecanismos de seguridad: autenticación y validación de entradas	Variables de entorno y validación
3.1.3	Aplica manejo de errores en el consumo de servicios	try, timeout y degradación
3.1.4	Ajusta la seguridad del código con apoyo de IA, documentando decisiones	La tabla de análisis

 Es una evaluación propia

- **ES3 · Unidad 3 · 25 %**
- Trabajo **individual** : no hay equipo
- **Sin defensa oral** : se corrige el script

LO QUE SE PREGUNTA EN LA DEFENSA

Dónde está la clave y por qué ahí · qué pasa si el servicio no responde · qué se hizo con lo que sugirió la IA.

ACTIVIDAD • INDIVIDUAL

60 minutos

Conectar el proyecto a un servicio

ENUNCIADO

Agregue al proyecto **un servicio externo**: el **clima**, para anticipar retrasos en los proyectos, o un **indicador económico**, para ajustar pagos internacionales. Son los dos que pide la guía. Con su clase cliente, su validación y su manejo de errores.

**Y DESPUÉS, CON INTELIGENCIA ARTIFICIAL**

Pida a una herramienta que revise el código del cliente. Anote los cinco aspectos de la lista y qué decidió en cada uno.

**Qué entregar**

- La clase cliente del servicio
- La función de validación
- El `.env.example` y `.gitignore` el
- La salida, incluida la de un fallo provocado
- La tabla de análisis de la IA

CIERRE DEL BLOQUE

Con qué se sale de esta parte

 Lo que queda listo

- Un servicio externo consumido de verdad
- La clave fuera del código y del repositorio
- Errores de red atrapados y degradación prevista
- La respuesta validada antes de guardarse

→ **Lo que viene**

- El proyecto integrador de la Unidad 4, en grupo
- Metodologías ágiles y trabajo con ramas
- El informe técnico y la defensa

✓ EL CAMBIO DE ESTA PARTE, EN UNA FRASE

El sistema dejó de estar solo. **Habla con otros servicios**, se identifica para hacerlo y desconfía de lo que recibe.

Lo que viene de afuera es sospechoso hasta validarlo

La red falla, los servicios cambian y las claves se filtran. Programar para eso es la diferencia entre un ejercicio y un sistema.

TI3V21 • Programación Orientada a Objeto Seguro • INACAP Valparaíso